

MoshSplit - Project Context

Project Vision

MoshSplit is a transparent shared-expense management app for friend groups, trips, festivals, and chaotic social events.

Primary use case:

- Group: Vira Latas
- Event: Wacken 2026

The app focuses on transparency, explainability, and trust.

This is NOT a Splitwise clone.

Core Philosophy

The application must always explain:

- Who owes whom
- How much
- Why
- Which expense created the debt
- How the debt changed over time
- Which payments modified the balance

The app must never perform hidden “magic” balance calculations.

Core Principles

- Expenses can change
- Payments are immutable
- Balances are computed

Balances are always recalculated from:

- Current expense state
 - Payment history
 - Settlements
-

Offline First PWA

MoshSplit MUST function as an offline-first PWA.

Users must be able to:

- Open the app without internet
- Create expenses offline
- Edit expenses offline
- View balances offline
- Queue changes locally
- Synchronize once internet becomes available

PWA functionality is a first-class requirement.

Monorepo Structure

```
moshsplit/
├── .opencode/
│   ├── agents/
│   │   ├── architect/
│   │   ├── backend-rust/
│   │   ├── frontend/
│   │   └── devops/
│   ├── memory/
│   │   ├── architecture/
│   │   ├── product/
│   │   ├── decisions/
│   │   └── experiments/
│   ├── AGENTS.md
│   └── context.md
├── apps/
│   ├── web/
│   └── pitboss-api/
├── docs/
├── infra/
│   ├── compose/
│   ├── docker/
│   ├── pulumi/
│   └── scripts/
├── packages/
├── scripts/
└── vendor/
```

Monorepo Philosophy

The project follows Eduardo's standard DDD-inspired monorepo architecture.

The `.opencode` directory is a first-class project citizen used for:

- agent orchestration
- architecture context
- development memory
- workflow coordination
- project intelligence

`packages/` is used to share internal reusable logic without publishing npm/cargo packages externally.

Vendor Directory

vendor/ must be gitignored.

This folder is used for externally downloaded projects or SDKs that are not publicly exposed.

Example:

- Sentinel SDK
- internal dependencies
- downloaded release artifacts

The goal is reproducible local development without exposing private dependencies publicly.

Dockerization Requirement

The entire application **MUST** be fully dockerized.

This includes:

- frontend
- backend
- database
- development environment
- hot reload workflows
- local tooling

The entire project must be runnable using:

docker compose up

Development containers must support:

- hot reload
 - reproducible environments
 - onboarding with minimal setup
 - local persistence
-

Backend Naming

Recommended backend service name:

pitboss-api

Reason:

“The pit boss controls the chaos.”

Sentinel Integration

Use the latest stable version of Sentinel.

Reference repository:

Sentinel is responsible for:

- authentication
- authorization
- JWT/session validation
- user management
- permission enforcement

MoshSplit consumes authenticated user context only.

Database Design

PostgreSQL should use two schemas:

auth/
Managed by Sentinel

app/
Managed by MoshSplit

The application must never mutate Sentinel-managed tables directly.

Sentinel remains the source of truth for:

- users
 - authentication
 - sessions
 - roles
 - permissions
-

Currency Philosophy

The accounting/base currency is EUR.

All calculations are performed in EUR cents.

Other currencies such as:

- BRL
- USD
- COP
- MXN

are cosmetic display estimates only.

Changing exchange rates must NEVER affect:

- balances
 - debts
 - settlements
 - payments
 - expense history
-

Architecture Principle

This is NOT an accounting application.

This is a transparent shared-expense coordination app for friend groups and trips.

The most important feature is:

Trust through explainability.
